Методы оптимизации
ИТМО, 2 курс, 4 семестр

Лекция №7
Модификации метода градиентного спуска, применяемые в нейросетевых вычислениях. Введение в нейронные сети

Запорожцев Иван Федорович
zaporozhtsev.if.work@gmail.com

Модификации метода градиентного спуска, применяемые в нейросетевых вычислениях. Введение в нейронные сети

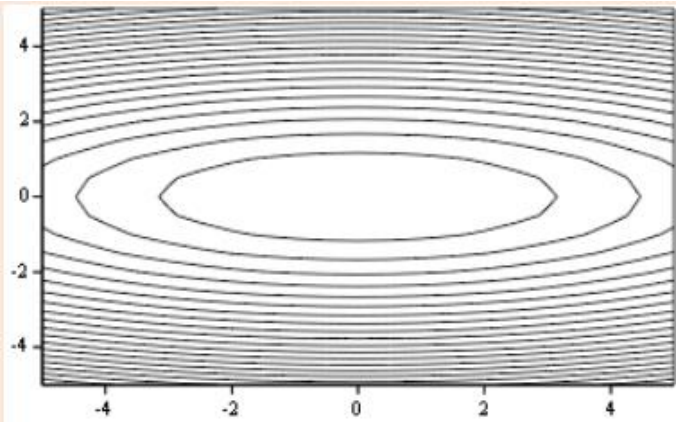
Часть I. Модификации метода градиентного спуска

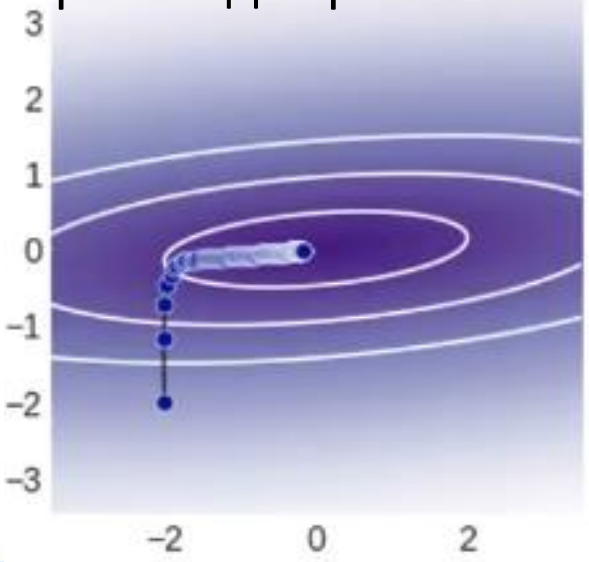
Градиентный спуск

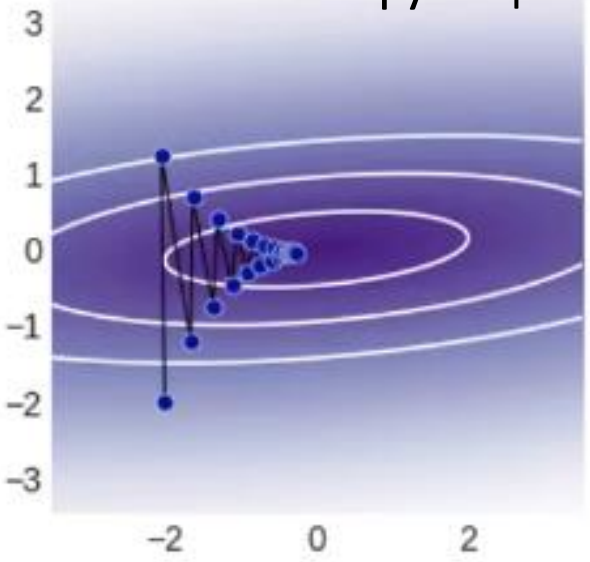
Замечания:

1.Метод сходится медленно в случае оврагов.

Вдоль направлений, характеризующих «дно оврага», «овражная» функция меняется незначительно, а в других направлениях, характеризующих «склон оврага», происходит резкое изменение значений функции.







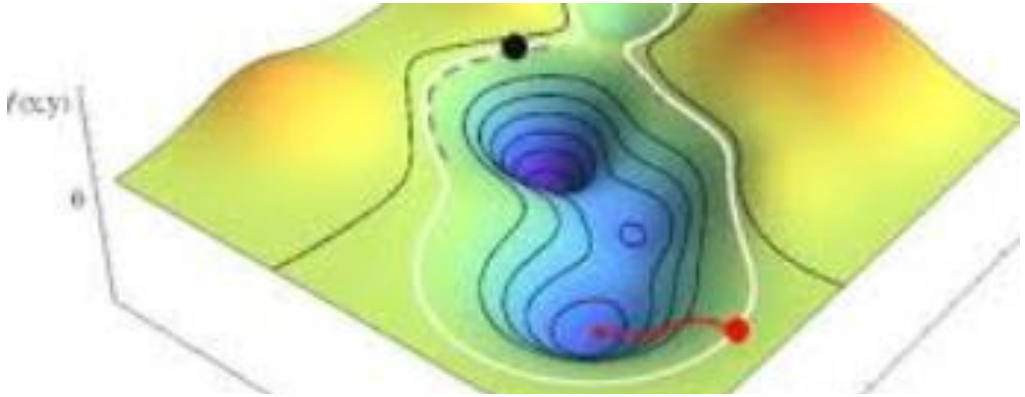
Масштабирование признаков в методах машинного обучения (или применить покоорд. спуск)

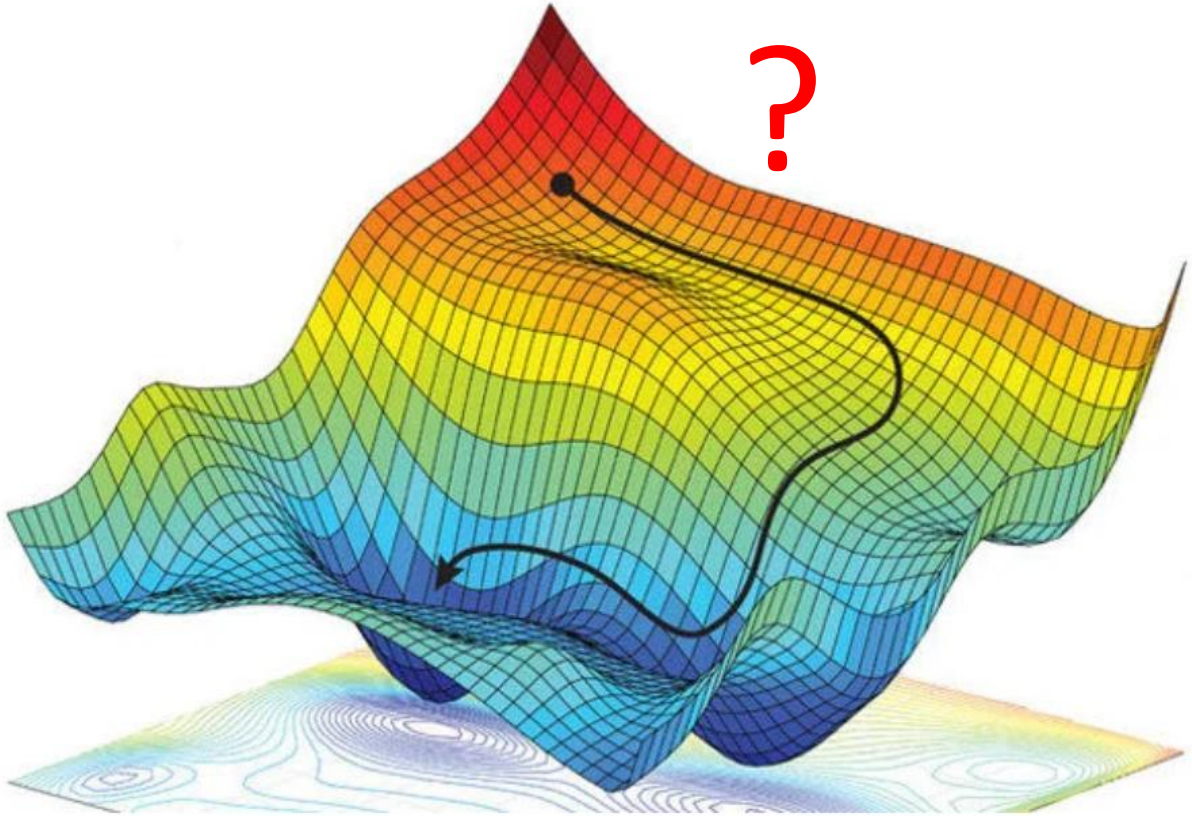
Градиентный спуск

Замечания:

2.Чувствителен к погрешностям вычислений (особенно сильно этот недостаток сказывается в малой окрестности точки минимума).

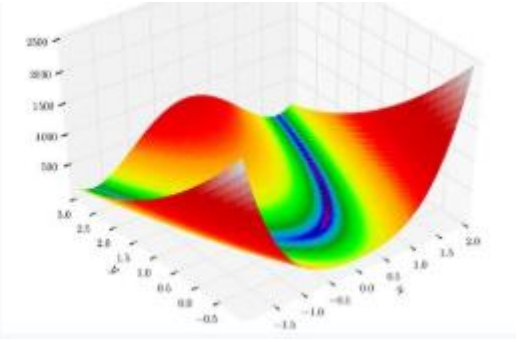
3.Может сходиться к седловой точке, «застревать в локальных минимумах», плохая сходимость для областей, где функция не явл. выпуклой.





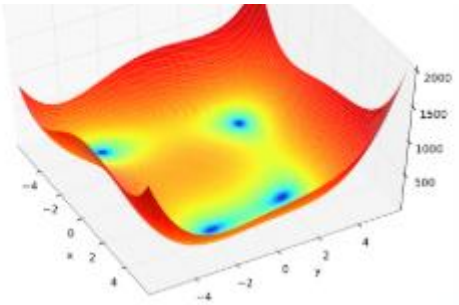
Сложные линии уровня функции потерь. «Искусственные ландшафты»

Функция Розенброка



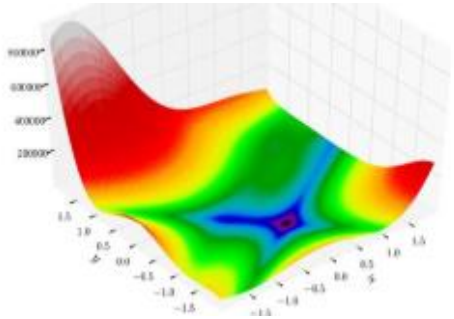
$$f(1,1)=0,$$

Функция Химмельблау



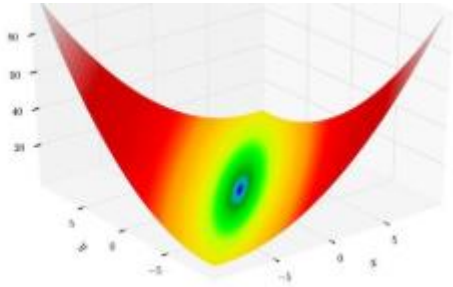
$$\text{Min} = \begin{cases} f(3.0, 2.0) & = 0.0 \\ f(-2.805118, 3.131312) & = 0.0 \\ f(-3.779310, -3.283186) & = 0.0 \\ f(3.584428, -1.848126) & = 0.0 \end{cases}$$

Функция Гольдшейна-Прайса



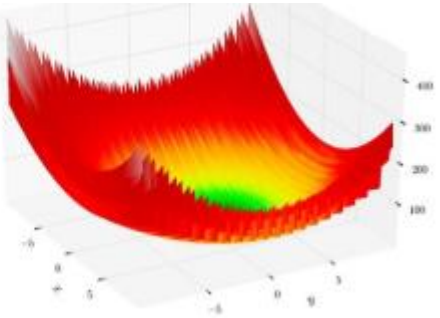
$$f(0,-1)=3$$

Функция Матьяса



$$f(0,0)=0$$

Функция Леви N 13



$$f(1,1)=0$$

Сложные линии уровня функции потерь. «Искусственные ландшафты»

Практическое замечание:

По числу обусловленности задачи можно судить о том, насколько линии уровня вытянуты в окрестности оптимума!

«Проблемы уровня данных»
Число обусловленности матрицы «объект-признак»

хотим решить $Xw = y$

Число обусловленности характеризует чувствительность решения СЛАУ при малом варьировании матрицы коэффициентов (чувствительность функции к изменению аргумента -- насколько может измениться значение функции при небольшом изменении аргумента).

Это проблема уровня данных – касается самой матрицы коэффициентов.

Больше число обусловленности – больше чувствительность – хуже для практики!

Чувствительность на уровне модели уменьшается за счёт **регуляризаторов** (вспомните L1 и L2: сделаем веса поменьше, чтоб каждое слагаемое $w_i x_i$ мало менялось при изменении x_i).

«Проблемы уровня данных»
Число обусловленности матрицы «объект-признак»

В линейной регрессии используется псевдообратная матрица Мура-Пенроуза (обобщение на случай неквадратной матрицы «объект-признак» X):

$$(X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}$$

хотим решить $Xw = y$

$$w = (X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}y$$

Проблема вырожденности и плохой обусловленности рассматривается для неё (большое число обусловленности *μ, определяемое по собственным числам матрицы*):

$$\mu(X^{\mathrm{T}}X) = \| X^{\mathrm{T}}X \| \cdot \| (X^{\mathrm{T}}X)^{-1} \| = \frac{\lambda_{\max}(X^{\mathrm{T}}X)}{\lambda_{\min}(X^{\mathrm{T}}X)}$$

«Проблемы уровня данных»
Число обусловленности матрицы «объект-признак»

Как улучшить обусловленность?

На уровне данных:

1) добавить новые данные в датасет, провалидировать имеющиеся.

На уровне модели:

1) регуляризация,
2) предобработка (исключение шума из признаков),
3) применить PCA для уменьшения кол-ва признаков (признаки как шум),
4) применить кастомизированную стратегию Feature Engineering (generation & selection).

Стохастический градиентный спуск. Обновление по батчам

Экономим ресурсы и не переобучаемся!

Повторять **<u>эпохи</u>** до выполнения критерия останова. Содержание одной эпохи:

1. Перемешать объекты обучающей выборки (перенумерация).
2. Рассматривать подвыборки из последовательно взятых *B* объектов (**<u>батчи</u>**), подвыборки не пересекаются, обновить веса градиентным спуском для каждого батча.

Т.к. работаем с подвыборкой, может оказаться и так (это всего лишь аппроксимация градиента):

Момент инерции (momentum)

Уменьшим эффект от градиента на текущей итерации (нужна память о предыдущих направлениях!)

Начнем с физической аналогии. Представим себе мячик, который катится с горы (гора – линии уровня функции потерь, траектория мячика – ломаная последовательных приближений).

Мячик не застрянет в небольшой ложбине, перелетит её в направлении низины, нужно приложить существенное усилие, чтоб поменять его траекторию. Что здесь работает: сумма сил, кинетическая энергия, импульс, движение по инерции.

Откуда название «момент»? От «момент инерции». При поступательном движении мера инерции – масса тела/материальной точки, при вращательном – момент инерции.

Момент инерции (momentum)

В строительном проектировании:

Момент инерции — это свойство сечения, используемое для определения способности элемента конструкции сопротивляться изгибающим и скручивающим силам. Обычно это довольно хороший показатель жесткости и прочности секций под нагрузкой.

Момент инерции для градиентного спуска

Требования («в идеале»): выпуклая гладкая функция в окрестности т. минимума

Модификация градиентного спуска (метод тяжелого шарика, метод Поляка):

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k) + \beta_k (x_k - x_{k-1}).$$

Модификация наискорейшего спуска (подбор скорости обучения):

$$(\alpha_k, \beta_k) = \arg\min_{\alpha,\beta} f(x_k - \alpha \nabla f(x_k) + \beta(x_k - x_{k-1})).$$

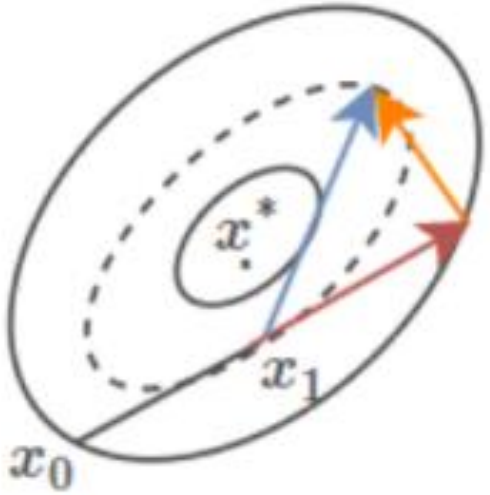
Метод Нестерова (NAGD, Nesterov Accelerated Gradient Descent)
– градиентный спуск из точки, которая продолжает предыдущее направление:

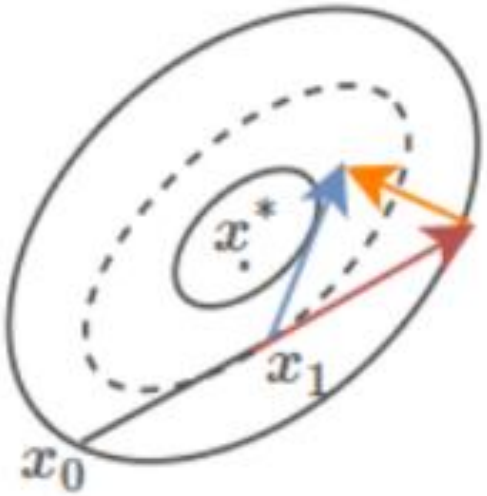
$$y_k = x_k + \beta\,(x_k - x_{k-1})\,, \qquad \text{momentum step}$$

$$x_{k+1} = y_k - \alpha \nabla f(y_k). \qquad \text{"lookahead" gradient step}$$

Момент инерции для градиентного спуска

Метод Поляка и метод Нестерова: обратите внимание, нормали к каким линиям уровня используются!



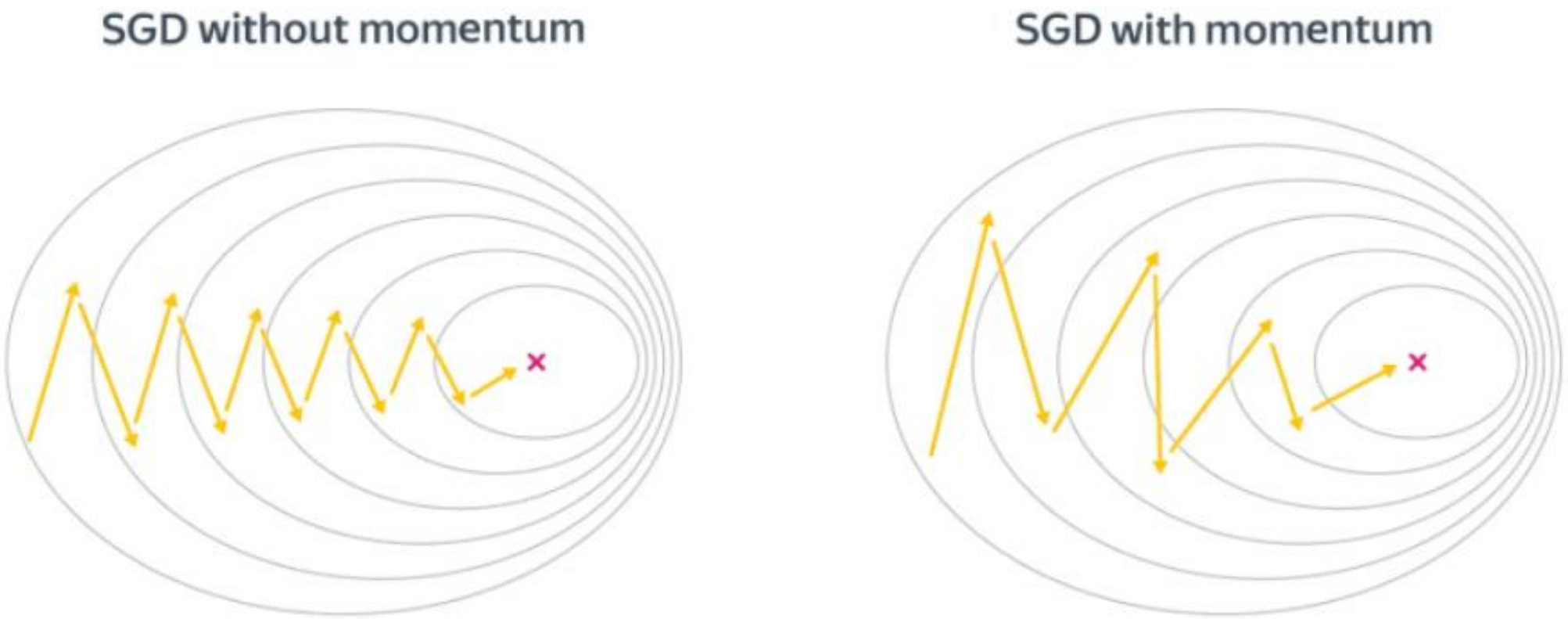


$$x_{t+1} = x_t - \alpha \nabla f(x_t) + \mu(x_t - x_{t-1})$$

$$x_{t+1} = x_t + \mu(x_t - x_{t-1}) - \gamma \nabla f(x_t + \mu(x_t - x_{t-1})$$

Момент инерции для SGD

Модификация с моментом позволяет уменьшать «паразитические» составляющие (в данном случае вертикальные, с ростом номера итерации) и для SGD.



Adagrad

Формулы итерационного процесса (адаптивная скорость обучения):

$$G_{k+1} = G_k + (\nabla f(x_k))^2$$

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{G_{k+1} + \varepsilon}} \nabla f(x_k).$$

Суть метода: возведение в квадрат и деления векторов покомпонентные; добавляем некоторую квазиньютоновость (за счёт квадратов) и начинаем динамически подбирать размер шага для каждой компоненты отдельно; если компонента стала затухать, то увеличить шаг, если расти, то уменьшить.

RMSProp

Формулы итерационного процесса (адаптивная скорость обучения):

$$G_{k+1} = \gamma G_k + (1-\gamma)(\nabla f(x_k))^2$$
$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{G_{k+1} + \varepsilon}} \nabla f(x_k).$$

Суть метода: учитывается история градиентов, но появляется «сглаживание в окне» (**простое экспоненциальное сглаживание**) – распределение единичного коэффициента как весов между двумя слагаемыми.

Эмпирическое замечание: скорость обучения уменьшается не так быстро, как для **Adagrad, потому считается более предпочтительным.**

Adam = ADAptive Momentum

Формулы итерационного процесса:

$$v_0 = 0, \quad m_0 - 0,$$

на итерации t с градиентом g получаем для вектова весов w :

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$

- 1-й момент: агрегат направлений (уменьшить вклад нового вектора градиента),

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

- 2-й момент: агрегат градиентов (сумма на истории) для скорости обучения (RMSProp, компенсирующий эффект для шага в направлении)

$$\hat{m}_t = \frac{1}{1-(\beta_1)^t} m_t, \quad \hat{v}_t = \frac{1}{1-(\beta_2)^t} v_t,$$

- нормализация (bias correction – обеспечение несмещенной оценки)

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{\hat{v}_t}} \hat{m}_t$$

Обычно β1 = 0.9 , β2 = 0.99. Требуется выбрать только базовую скорость обучения *α*. Иногда начинают с *числа Андрея Карпаты* (пост 2016 года) = 3e-4.

Adam = ADAptive Momentum. Нормализация (bias correction)

Авторы оригинальной статьи указывали, что 1-й и 2-й моменты должны быть несмещенными (bias -- смещение) оценками M(g) и M(g^2), где g – вектор градиент:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

$$\hat{m}_t = \frac{1}{1-(\beta_1)^t} m_t, \quad \hat{v}_t = \frac{1}{1-(\beta_2)^t} v_t,$$

Adam = ADAptive Momentum. Нормализация (bias correction)

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t =$$

$$= \beta_1 \left(\beta_1 m_{t-2} + (1-\beta_1) g_{t-1} \right) + (1-\beta_1) g_t =$$

$$= \beta_1 \left(\beta_1 \left(\beta_1 m_{t-3} + (1-\beta_1) g_{t-2} \right) + (1-\beta_1) g_{t-1} \right) + (1-\beta_1) g_t =$$

$$= \beta_1 \left(\beta_1^2 m_{t-3} + \beta_1 (1-\beta_1) g_{t-2} + (1-\beta_1) g_{t-1} \right) + (1-\beta_1) g_t =$$

$$= \beta_1^3 m_{t-3} + \beta_1^2 (1-\beta_1) g_{t-2} + \beta_1 (1-\beta_1) g_{t-1} + (1-\beta_1) g_t =$$

$$= 0 + (1-\beta_1) \sum_{s=1}^{t} \beta_1^{t-s} g_s$$

$$M(m_t) = M\left((1-\beta_1) \sum_{s=1}^{t} \beta_1^{t-s} g_s \right) = (1-\beta_1) \cdot M\left(\sum_{s=1}^{t} \beta_1^{t-s} g_s \right) = \left\{ \begin{matrix} g_s - \text{одинаково} \\ \text{распределенные СВ} \end{matrix} \right\} =$$

$$= \left((1-\beta_1) \cdot \sum_{s=1}^{t} \beta_1^{t-s} \right) \cdot M(g_s) = \left\{ \begin{matrix} \text{формула сокращенного} \\ \text{умножения} \end{matrix} \right\} = \left(1-\beta_1^t\right) \cdot M(g_s)$$

– имеем смещенную оценку матожидания градиента…

Weight Decay (затухание весов)

Суть:

(1) ввести в функцию потерь регуляризатор со смыслом «маленький по модулю вес – это хорошо»,

(2) на каждом шаге уменьшать вес пропорционально коэффициенту регуляризации

L2 regularization for the vanilla gradient descent algorithm

We penalize the Loss function

$$\tilde{L}(\theta) = L(\theta) + \boxed{\frac{\lambda}{2}\|\theta\|_2^2}$$

L2 regularization term

Then we update the model parameters with the gradient of the Loss function

$$\boxed{\theta_t} = \boxed{\theta_{t-1}} - \boxed{\alpha \nabla_\theta \tilde{L}(\theta_{t-1})} = \theta_{t-1} - \alpha \nabla_\theta L(\theta_{t-1}) \boxed{- \alpha\lambda\theta_{t-1}}$$

New value

Old value

gradient update

The L2 regularization acts as a weight decay mechanism

AdamW (Adam с затуханием весов)

L2 regularization for Adam

We penalize the Loss function

$$\tilde{L}(\theta) = L(\theta) + \boxed{\frac{\lambda}{2}\|\theta\|_2^2}$$

We compute 1st and 2nd moments

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta \tilde{L}(\theta_{t-1})$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta \tilde{L}(\theta_{t-1}))^2$$

We normalize

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

We update the model parameters

$$\theta_t = \theta_{t-1} - \boxed{\frac{\alpha}{\sqrt{\hat{v}_t}}\hat{m}_t}$$

The L2 regularization DOES NOT acts as a weight decay mechanism

L2 regularization for AdamW

We DO NOT penalize the Loss function

We compute 1st and 2nd moments

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_\theta L(\theta_{t-1})$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_\theta L(\theta_{t-1}))^2$$

We normalize

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

We update the model parameters and ADD the regularization term

$$\theta_t = \theta_{t-1} \boxed{- \alpha\lambda\theta_{t-1}} - \frac{\alpha}{\sqrt{\hat{v}_t}}\hat{m}_t$$

The L2 regularization DOES acts as a weight decay mechanism

$\Longrightarrow$ Training more stable and better generalization with AdamW with L2 regularization

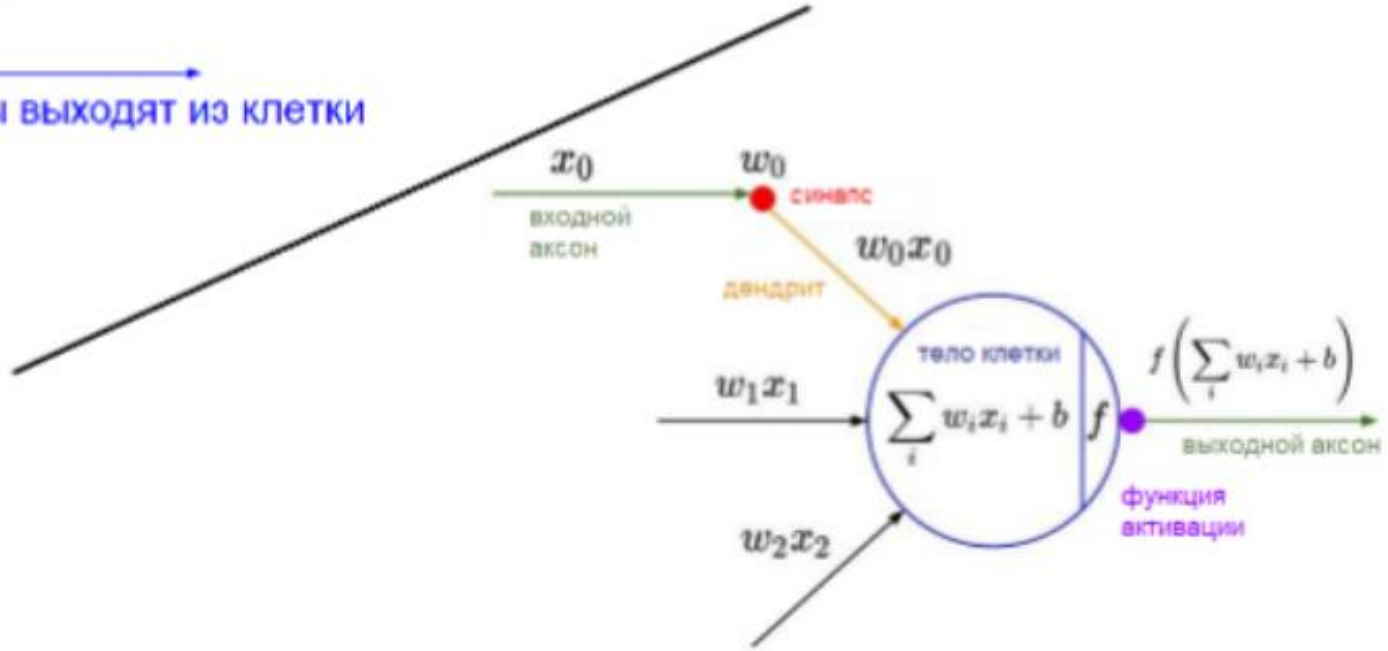
Модификации метода градиентного спуска, применяемые в нейросетевых вычислениях. Введение в нейронные сети

Часть II. Введение в нейронные сети

Вехи развития нейросетевых вычислений

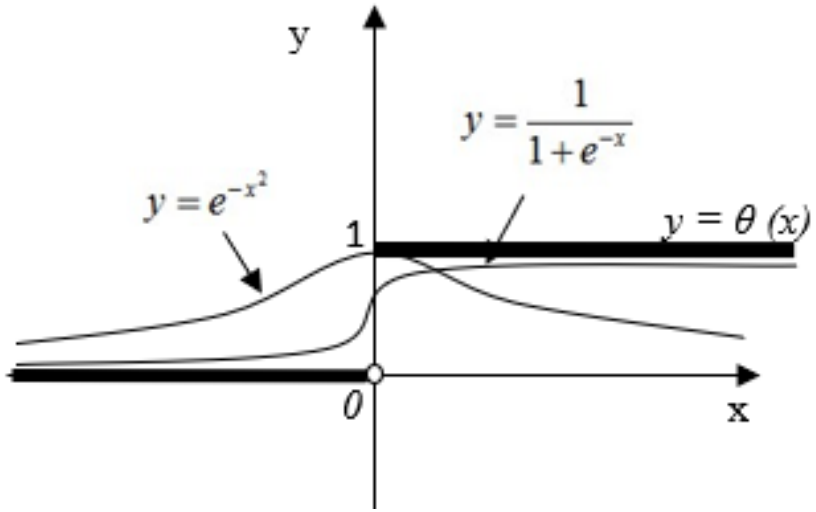
- **1943** - *Warren McCulloch*, *Walter Pitts* формируют понятие нейронной сети.
- **1958-1960** - *Frank Rosenblatt:* перцептрон и первый нейрокомпьютер
- **1974** - *Paul Werbos, Александр Галушкин* изобретают метод обратного распространения ошибки.
- **1986** - *Rina Dechter* формулирует понятие глубокого обучения -- Deep Learning.
- **1988-1989** - *Yann LeCun* предлагает концепцию сверточных нейронных сетей, а также использует метод обратного распространения ошибки для обучения глубокой сети.
- **200x** – Deep Learning (DL) активно замещает собой другие методы машинного обучения
- **201x** – Архитектуры в DL, придуманные для одних предметных областей, применяются к другим, переходя в разряд универсальных методов. Понятия Transfer Learning, Style Transfer. Бурное развитие CV-решений и генеративных методов.
- **202x** – Мультимодальность, ИИ-агенты: разные представления информации для одной модели – модель обучается / может выводить данные разного типа (текст, звук, изображения, табличные данные). Акцент на текст. Активное развитие LLM

Модель МакКаллока-Питтса
Нелинейность от линейной комбинации над входом

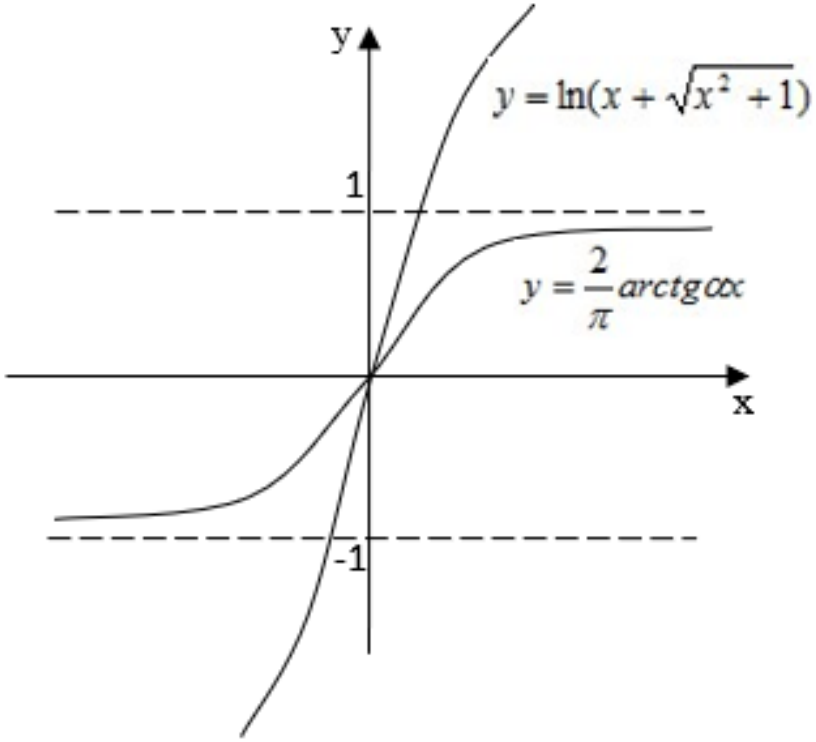


Функции активации

Униполярные

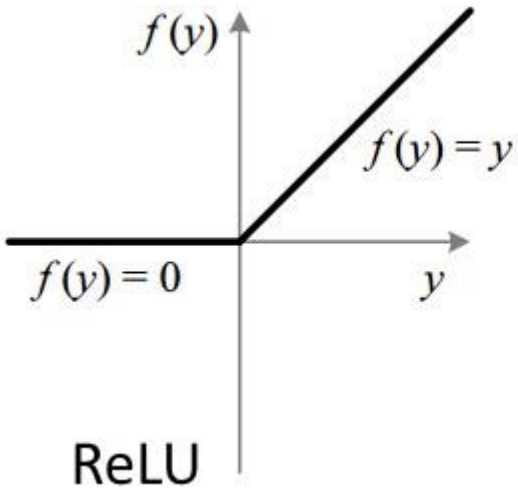


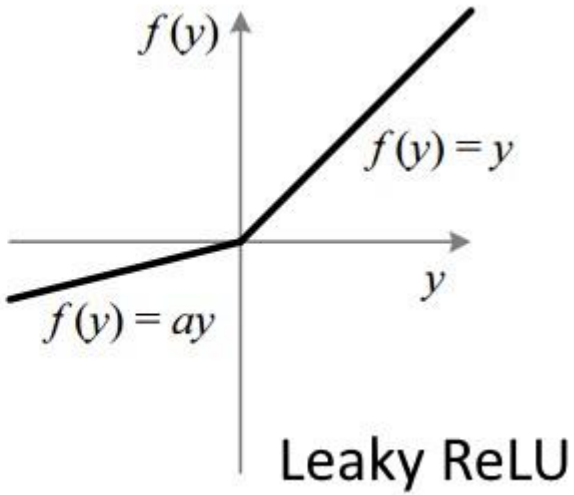
Биполярные



Ещё: $y = th\alpha x = \frac{e^{\alpha x} - e^{-\alpha x}}{e^{\alpha x} + e^{-\alpha x}}$

Функции активации. RELU и Leacky RELU





https://neerc.ifmo.ru/wiki/index.php?title=%D0%9F%D1%80%D0%B0%D0%BA%D1%82%D0%B8%D0%BA%D0%B8_%D1%80%D0%B5%D0%B0%D0%BB%D0%B8%D0%B7%D0%B0%D1%86%D0%B8%D0%B8_%D0%BD%D0%B5%D0%B9%D1%80%D0%BE%D0%BD%D0%BD%D1%8B%D1%85_%D1%81%D0%B5%D1%82%D0%B5%D0%B9

Многоклассовая логистическая регрессия

Multiclass logreg / multinomial regression

Для j-го класса имеем свою функцию $w_j^T x$
хотим такие значения превращать в распределения

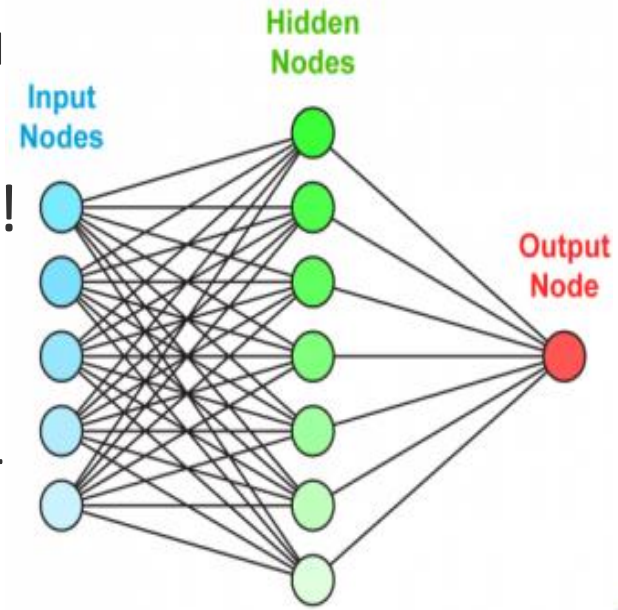
$$(w_1^T x, w_2^T x, \ldots, w_l^T x) \in \mathbb{R}^l \to (p_1(x), p_2(x), \ldots, p_l(x)) \in [0,1]^l_{\Sigma=1}$$

Такая функция называется softmax:

$$\mathrm{softmax}(a_1, \ldots, a_l) = \frac{1}{\exp(a_1) + \ldots + \exp(a_l)} [\exp(a_1), \ldots, \exp(a_l)]$$

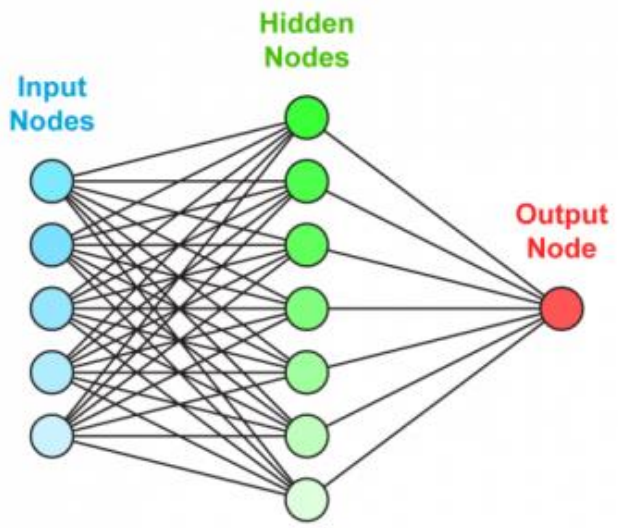
Fully Connected Layer (FC, Linear). Multilayer Perceptron

1. Нейрон текущего слоя имеет связь СО ВСЕМИ нейронами следующего слоя (Fully Connected, FC, Linear).
2. Первый слой (входной) не считается слоем (по сути слой характеризуется наличием входа и выхода, потому сам вход не слой) – на рисунке ДВУХСЛОЙНЫЙ ПЕРСЕПТРОН!
3. Основа вычислений для слоя (для каждого нейрона): скалярное произведение (линейная комбинация) + нелинейная ф-я (функция активации). Обучаемые веса – коэффициенты линейной комбинации.
4. Совокупность таких слоёв называется Multilayer Perceptron (MLP) – частный случай Feedforward Network (сеть прямого распространения, без циклов).

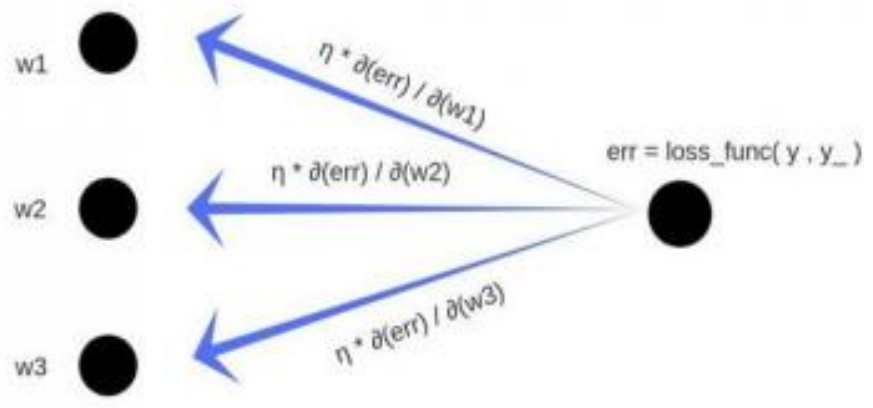


Fully Connected Layer (FC, Linear). Multilayer Perceptron

- Входной слой;
- Скрытые (вычислительные) слои;
- Выходной слой.



Обратное распространение

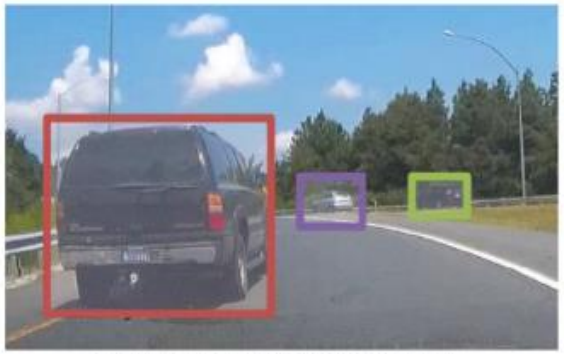


$$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^{(t)})$$

Что такое функция сети и как обновлять веса?
См. Доп. материал Метод Backpropagation

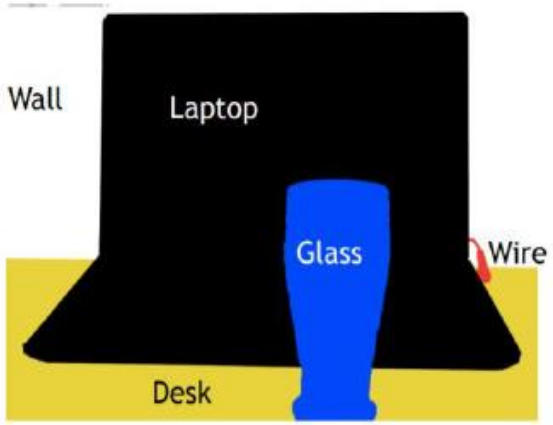
Convolutional Neural Network (CNN). Компьютерное зрение

Современные задачи



This image is licensed under CC BY-NC-SA 2.0; changes made

- Object detection
- Action classification
- Image captioning
- …



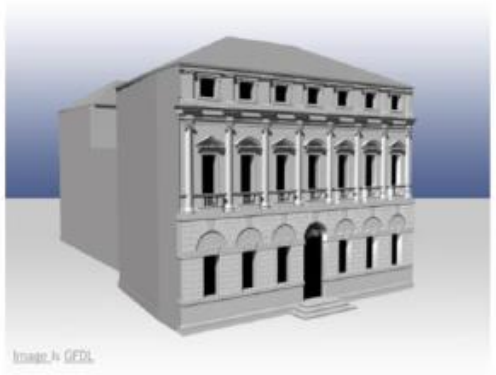
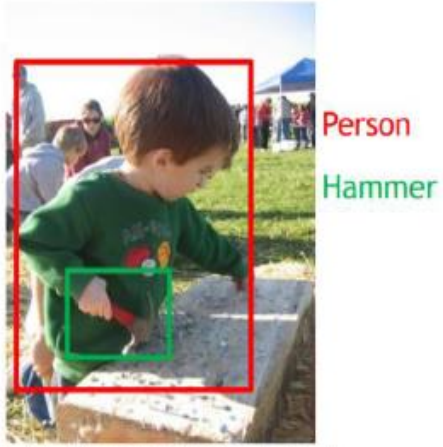
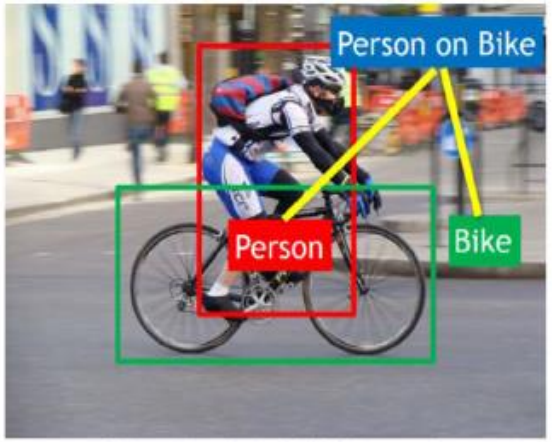


Image is GFDL



This image is licensed under CC BY-SA 2.0; changes made



This image is licensed under CC BY-SA 3.0; changes made

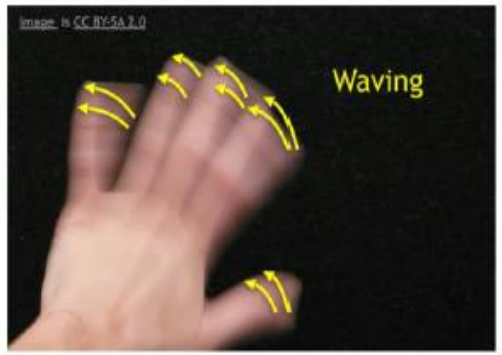


Image is CC BY-SA 2.0

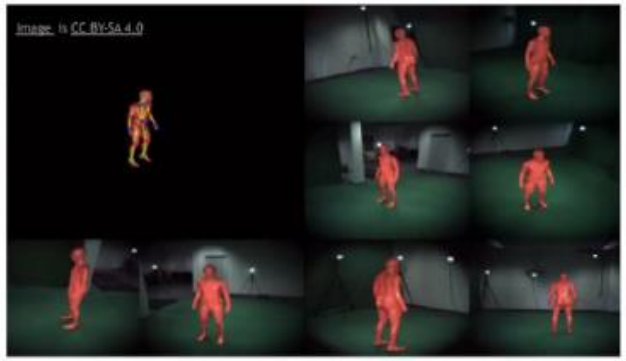
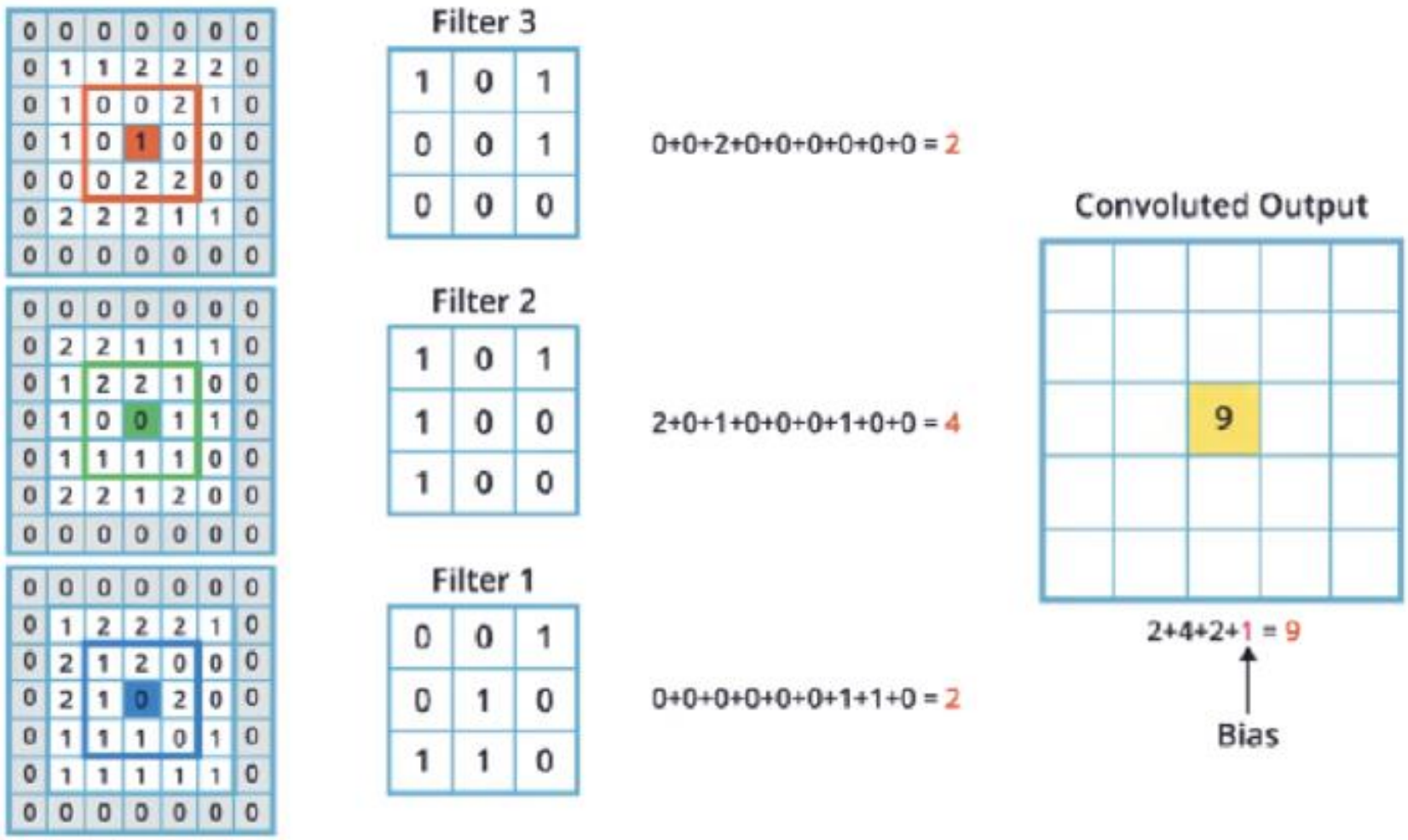


Image is CC BY-SA 4.0

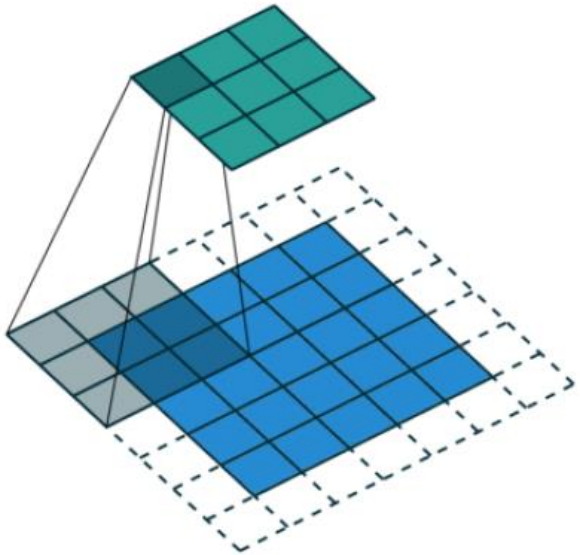
Преобразование «свёртка»

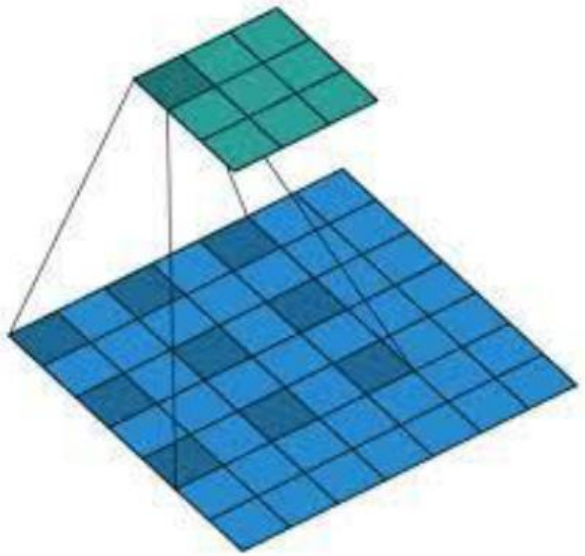


Преобразование «свёртка»

Дополнительные параметры свертки

- Stride
- Padding
- Dilation





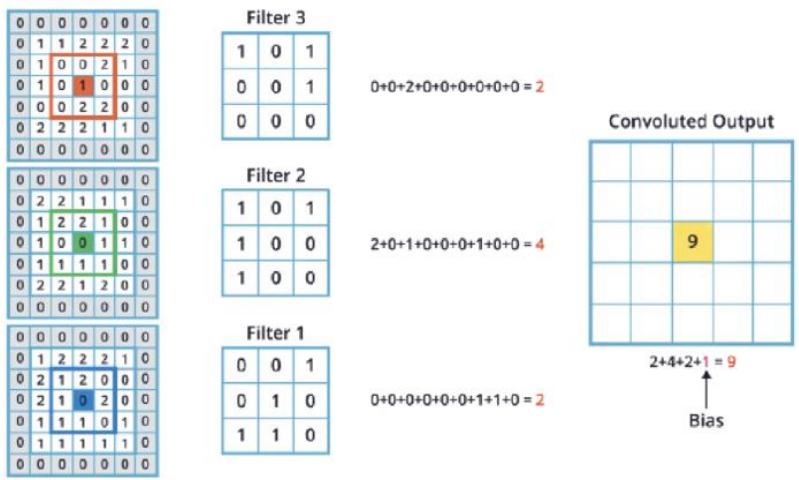
$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

Фильтр

Что такое фильтр?

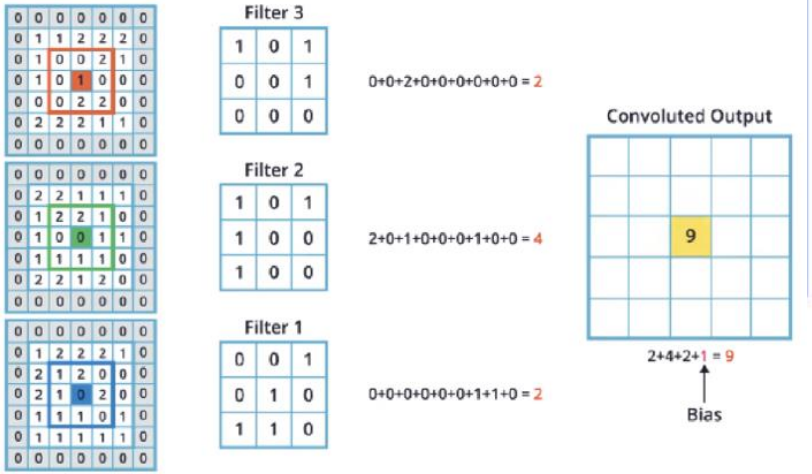
1. Квадратная матрица (окно => размер окна = порядок матрицы)
2. Пустая / непустая (числовые коэффициенты)
3. Обновляемые / необновляемые коэффициенты
4. Функция, которая применяется к окрестности пикселя, на которое наложено окно (оконное преобразование) – например, свёртка.



В каждом канале своя матрица коэффициентов!

Convolutional Layer

1.Когда говорят, что фильтр размером 5x5, то он размером 5 x 5 x «Кол-во каналов изображения».
2.Смещение (вес “bias”) обычно игнорируют.
3.Обучаемые веса – это коэффициенты фильтра (по всем каналам!).



4. Что считать нейроном: однократное оконное преобразование свертки (по всем каналам), которое порождает псевдопиксель одного канала!
5. Что в основе вычислений? Свёртка (скалярное произведение в окне).
6. Что такое слой (Convolutional Layer): это совокупность сверточных фильтров с одинаковыми размерами и разными коэффициентами, применяемыми к одному изображению/псевдоизображению (Feature Map, карта признаков). Число каналов выходного псевдоизображения (Feature Map) = число фильтров.
7. «Нейрон» здесь не получает на вход всё изображение – не Fully Connected! (термин нейрон для данного слоя обычно и не используется!)

Convolutional Layer. Другие фильтры

Формирование линейного фильтра:

- Задать матрицу размерами m x n
- Выбрать коэффициенты в зависимости от ожидаемой операции
- Пример усредняющего фильтра 3 x 3: $R = \frac{1}{9}\sum_{i=1}^{9} z_i$.

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

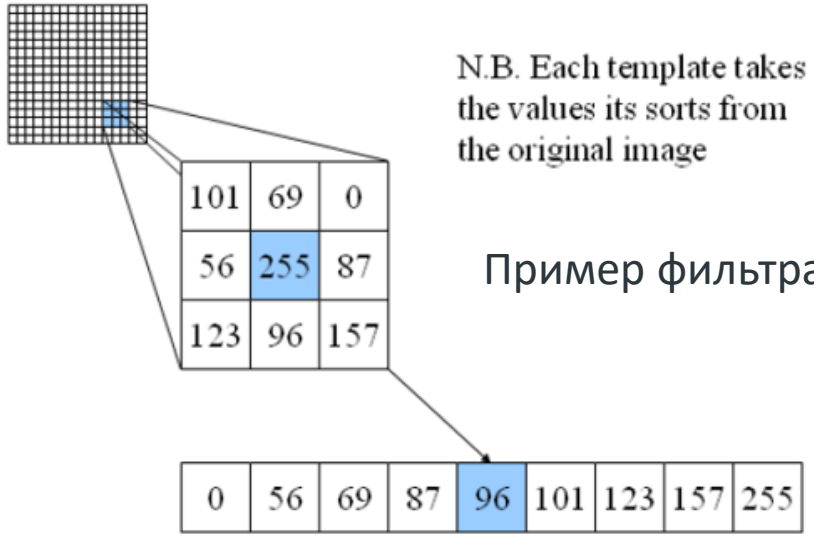
Пример фильтра:

- Взвешенное среднее

$$\frac{1}{16} \times \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

У этого фильтра веса фиксированы, не участвует в обучении => не для Convolutional Layer

Формирование нелинейного фильтра:



N.B. Each template takes the values its sorts from the original image

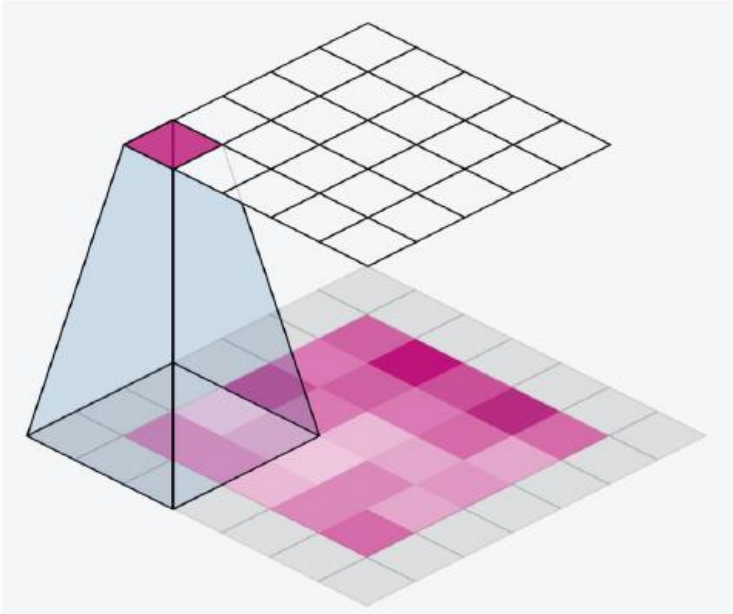
Пример фильтра: медианный фильтр

У этого фильтра веса отсутствуют, не участвует в обучении, функция не свёртка => не для Convolutional Layer

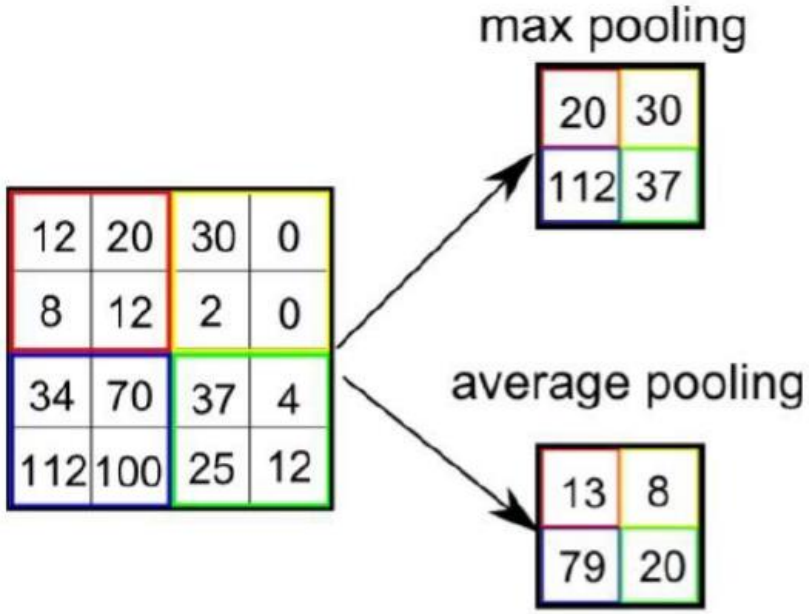
Convolutional Neural Network. Слой пуллинга (Pooling Layer)

Pooling (Subsampling)

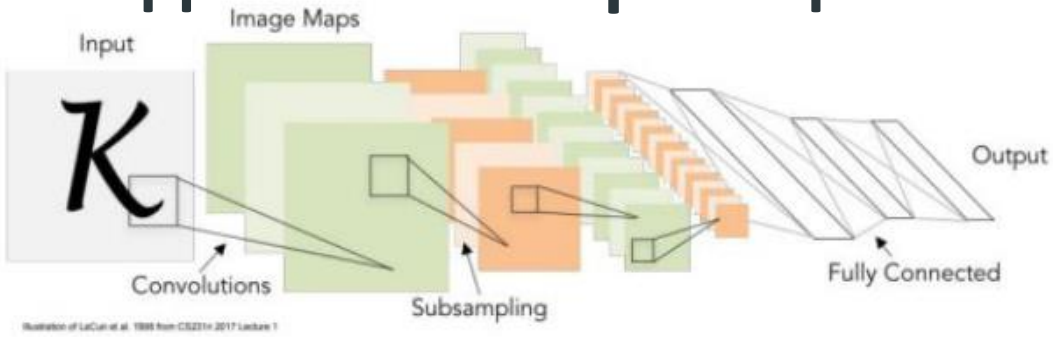
1. Уменьшить размерность
2. Выделить доминирующие признаки (шумоподавление)
3. Добавляет инвариантность к небольшим смещениям
4. Stride также доступен.



Как и для медианного фильтра: здесь веса отсутствуют, не участвует в обучении, функция не свёртка => min, max или average.



Convolutional Neural Network.
Задача классификации изображений



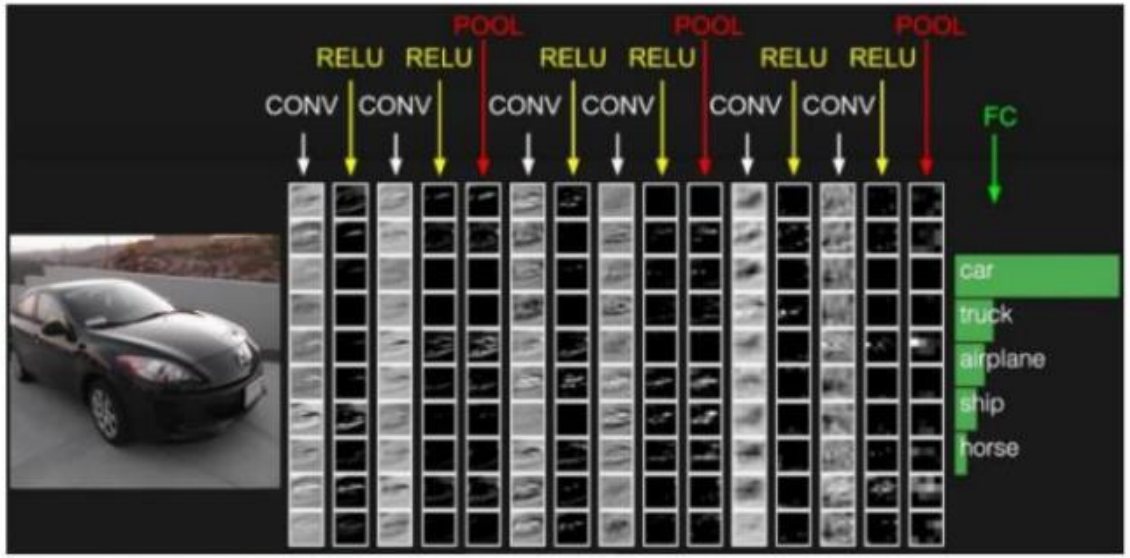
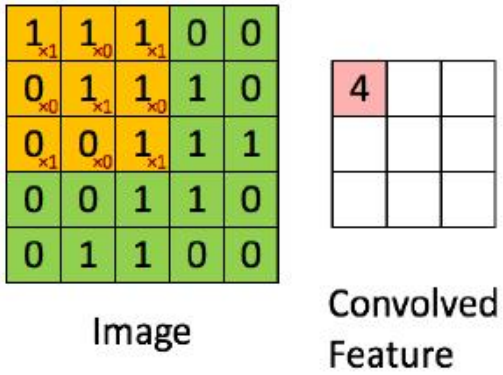


Схема работы свёрточной нейросети

См. пример подсчета кол-ва весов: Подсчет весов

Example Architecture: Overview. We will go into more details below, but a simple ConvNet for CIFAR-10 classification could have the architecture [INPUT - CONV - RELU - POOL - FC]. In more detail:

- INPUT [32x32x3] will hold the raw pixel values of the image, in this case an image of width 32, height 32, and with three color channels R,G,B.
- CONV layer will compute the output of neurons that are connected to local regions in the input, each computing a dot product between their weights and a small region they are connected to in the input volume. This may result in volume such as [32x32x12] if we decided to use 12 filters.
- RELU layer will apply an elementwise activation function, such as the $max(0, x)$ thresholding at zero. This leaves the size of the volume unchanged ([32x32x12]).
- POOL layer will perform a downsampling operation along the spatial dimensions (width, height), resulting in volume such as [16x16x12].
- FC (i.e. fully-connected) layer will compute the class scores, resulting in volume of size [1x1x10], where each of the 10 numbers correspond to a class score, such as among the 10 categories of CIFAR-10. As with ordinary Neural Networks and as the name implies, each neuron in this layer will be connected to all the numbers in the previous volume.

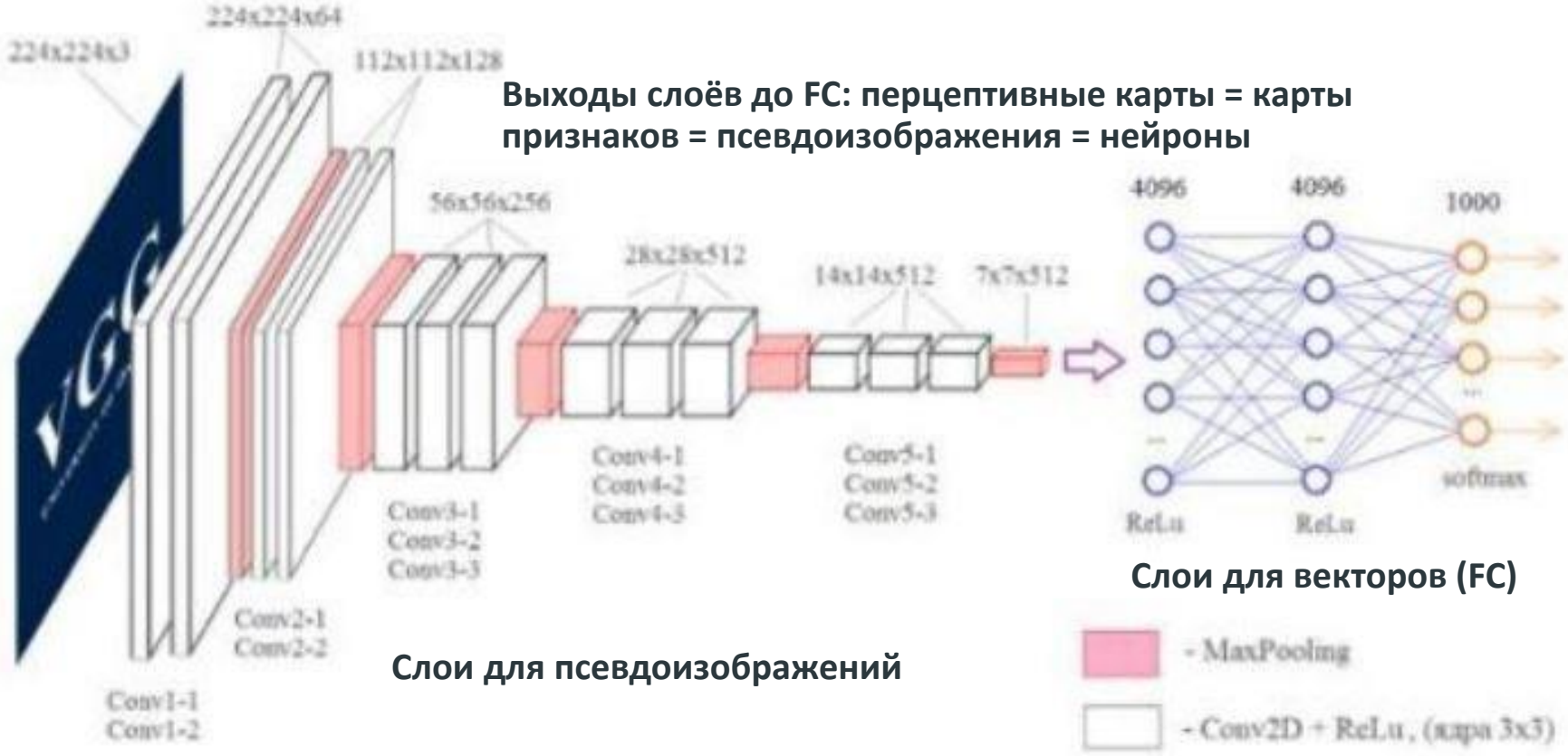


Convolutional Neural Network.
Задача классификации изображений

Решаем задачу классификации изображений:
1)выходной слой – нейроны классов, функция активации – softmax,
2)выход – скоры (вероятности) отнесения к классам

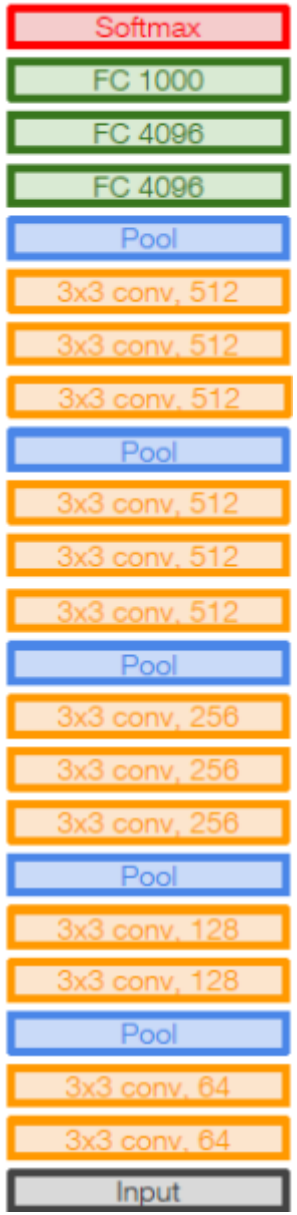
VGGNet

Выходы слоёв до FC: перцептивные карты = карты признаков = псевдоизображения = нейроны



Слои для векторов (FC)

Слои для псевдоизображений



Обучение с PyTorch. Стартовый пример

```
import torch

n_iter = 100
learning_rate = 0.3
step_x = 0.1
step_y = 0.1
bounds = np.asarray([[-7.0, 7.0], [-10.0, 41.0]])
history = {}

solution = torch.tensor([[-5.0], [40.0]], requires_grad=True)
optimizer = torch.optim.Adam([solution], lr=learning_rate)
trace = []

for i in range(n_iter):
    optimizer.zero_grad()
    score = objective(solution[0], solution[1])
    score.backward()
    trace.append(solution.clone().detach().numpy())
    optimizer.step()
for it in range(len(trace)):
  history[it] = {'solution': (trace[it][0,0], trace[it][1,0]), 'score': None}

plot_history('Adam_history', objective, bounds, step_x, step_y, history)
```

Спасибо за внимание!

Запорожцев Иван Федорович
zaporozhtsev.if.work@gmail.com